

Çizgeler ve Dolaşmalar

CEN207 Veri Yapıları — Hafta 5

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	Neden çizgeler, terimler Anim 1 · matris/liste Anim 2–3
2	BFS Anim 4 · özyinelemeli DFS Anim 5 · yinelemeli DFS Anim 6
3	Bağlı bileşenler Anim 7 · en kısa yol Anim 8

Öğrenme çıktıları: LO.1 (temel veri yapılarını açıklar) · LO.2 (karmaşıklığı çözümler) · LO.7 (doğru yapıyı seçer)

Bu haftanın konuları — nerede

Konu	Nerede
Neden çizgeler, terimler	Bölüm 1–2
Komşuluk matrisi ve komşuluk listesi	Bölüm 3
BFS, DFS (özyinelemeli ve yinelemeli)	Bölüm 4–6
Bağlı bileşenler, en kısa yol	Bölüm 7–8

Kod örnekleri nasıl çalışır

- Her fikrin eksiksiz bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-05/c/` ve `code/week-05/java/`
- Her programın beklenen çıktısı hafta notlarında var

Hatırlatma — yığın, kuyruk, özyineleme (Hafta 3)

- Bir **yığın** (stack, LIFO): son eklenen, ilk çıkarılan
- Bir **kuyruk** (queue, FIFO): ilk eklenen, ilk çıkarılan
- **Özyineleme** (recursion) aslında bir yığındır: çağrı ekler, dönüş çıkarır
- Bugün: aynı yığın ve kuyruk, artık çizge düğümleri tutuyor

Hatırlatma — ağaçlar özel çizgelerdir (Hafta 4)

- Bir ağaç: tek kök, döngü yok, herhangi bir düğüme tam bir yol
- Bir çizge: kök **zorunlu değil**, döngüye izin var, birden çok yol olabilir
- Her ağaç bir çizge**dir**; her çizge bir ağaç değildir
- Dolaşmalar (ön/orta/son/seviye sırası) bugün çizge aramasına genelleşiyor

Haftanın haritası — genel bakış

Temeller	Dolaşma ve uygulamalar
Neden çizgeler, terimler	BFS, DFS (özyinelemeli, yinelemeli)
Matris ve liste	Bağlı bileşenler
—	En kısa yol (ağırlıksız)

1. Neden Çizgeler? Köprülerden Ağlara

Başlangıç sorusu

Bir şehirde yedi köprüyle birbirine bağlı dört kara parçası var. Şehri, her köprüden tam bir kez geçerek dolaşıp eve dönebilir misiniz?

Kısa bir tarihçe

- **1736** — Leonhard Euler, Königsberg köprü bulmacasını çözer
- Königsberg (bugün Kaliningrad): 4 kara parçası, 7 köprü
- Euler'in makalesi yazılmış **ilk** çizge teorisi makalesidir
- Yanıt "hayır" oldu -- ve nedenini, her şehir için, tam olarak açıkladı

Köprüler, yeniden ifade edilirse

- Her **kara parçası** bir **düğüm** (vertex) olur (4 tane)
- Her **köprü** bir **kenar** (edge) olur (7 tane)
- Yürüyüş şuna dönüşür: kalemi kaldırmadan her kenarı bir kez çiz
- Buna **Euler yolu** (Eulerian path) denir -- Euler'in kendi adından

Euler'in içgörüsü: derece paritesi

- Bir düğümden **geçerken** her seferinde 2 kenar kullanılır
- Yalnızca başlangıç ve bitiş düğümü **tek** sayıda kenar kullanabilir
- Bir Euler yürüyüşü için **0 ya da 2** tek dereceli düğüm gerekir, fazlası değil
- Königsberg'de **4** tek dereceli düğüm var -- o yüzden yürüyüş yok

Neden önemli

- Sayarak çözülen bir bulmaca, **çizge teorisini** kurdu
- Aynı düğüm/kenar fikri artık neredeyse her ilişkiyi modelliyor
- Modern kullanımlar: haritalar, sosyal ağlar, web ve çok daha fazlası
- Sonraki üç slayt bunlardan üçünü gösteriyor

Çizgeler her yerde — yol haritaları

- **Düğüm**ler: kavşaklar, şehirler, adresler
- **Kenar**lar: yollar, bir **ağırlık**la -- mesafe ya da sürüş süresi
- GPS yönlendirmesi sorar: A'dan B'ye en kısa ağırlıklı yol nedir?
- Hafta 9'daki Dijkstra algoritması tam olarak bu soruyu yanıtlıyor

Çizgeler her yerde — sosyal ağlar

- **Düğüm**ler: kişiler ya da hesaplar
- **Kenar**lar: "takip ediyor" (yönlü) ya da "arkadaş" (yönsüz)
- "Arkadaşımdın arkadaş" -- tam olarak 2 kenarla erişilebilen
- Bölüm 4'teki BFS bu soruyu, bir seviye seviye yanıtıyor

Çizgeler her yerde — web

- **Düğüm**ler: web sayfaları; **kenar**lar: hiper bağlantılar (yönlü)
- $A > B$, **A** sayfasının **B** sayfasına bağlandığı anlamına gelir
- Bir tarayıcı (crawler) erişilebilir her sayfayı tam bir kez ziyaret eder
- Bölüm 6'daki DFS, bir web tarayıcısı yazmanın klasik yollarından biri

Çizge ve ağaç, yeniden

Ağaç (Hafta 4)	Çizge (bu hafta)
Tek kök	Zorunlu kök yok
Döngüye izin yok	Döngüye izin var
İki düğüm arasında tam bir yol	Sıfır, bir ya da çok yol

Sık yapılan hatalar

- Bir çizgenin **bağlı** olması gerektiğini varsaymak -- olmayabilir
- Bir kenarın her zaman **iki yönde** çalıştığını varsaymak -- yalnız yönsüzse
- Hiç kenarı olmayan tek bir düğümün geçerli bir çizge olduğunu unutmak

Mini soru

Bir şehirde tam olarak **iki** tek dereceli kara parçası var, geri kalanı çift. Her köprüden bir kez geçen bir yürüyüş var mı? Nereden başlamalı?

Yanıt

Evet. Tam olarak iki tek dereceli düğümle, bir Euler **yolu** (tam bir tur değil) vardır; bir tek düğümden başlar, diğerinde biter.

2. Çizge Terimleri

Başlangıç sorusu

Bir çizge yalnızca iki kümedir: düğümlerin kümesi **V**, kenarların kümesi **E**. Bu kadar küçük bir fikirden gerçekten ne kadar yapı çıkar?

Bir çizge, biçimsel olarak

- Bir çizge $G = (V, E)$: bir düğüm kümesi, bir kenar kümesi
- Her kenar **iki** düğümü bağlar (aynı düğüm de olabilir)
- $|V|$ düğüm, $|E|$ kenar -- kısaca: V ve E
- Aşağıdaki her şey yalnızca bu iki kümeden inşa edilir

Terimler (1/4)

Terim	Anlamı
Düğüm (vertex)	Çizgedeki tek bir nokta
Kenar (edge)	İki düğüm arasındaki bağlantı
Yönsüz (undirected)	A-B kenarı her iki yönde kullanılabilir
Yönlü (directed)	A>B kenarı yalnız A 'dan B 'ye kullanılabilir

Terimler (2/4)

Terim	Anlamı
Ağırlıklı (weighted)	Her kenar bir sayı, ağırlığını taşır
Ağırlıksız (unweighted)	Kenarlar yalnızca "bağlı" der, sayı yok
Derece (degree, yönsüz)	Bir düğüme değen kenar sayısı

Terimler (3/4)

Terim	Anlamı
Gelen derece (in-degree)	Bir düğüme giren kenarlar (yönlü)
Giden derece (out-degree)	Bir düğümden çıkan kenarlar (yönlü)
Yol (path)	Düğüm tekrarsız bir kenar dizisi
Döngü (cycle)	Başladığı düğüme dönen bir yol

Terimler (4/4)

Terim	Anlamı
Bağlı (connected)	Her düğüme her düğümden erişilebilir
Öz-döngü (self-loop)	Bir düğümden kendisine giden kenar
Çoklu kenar (multi-edge)	Aynı çift arasında birden fazla kenar

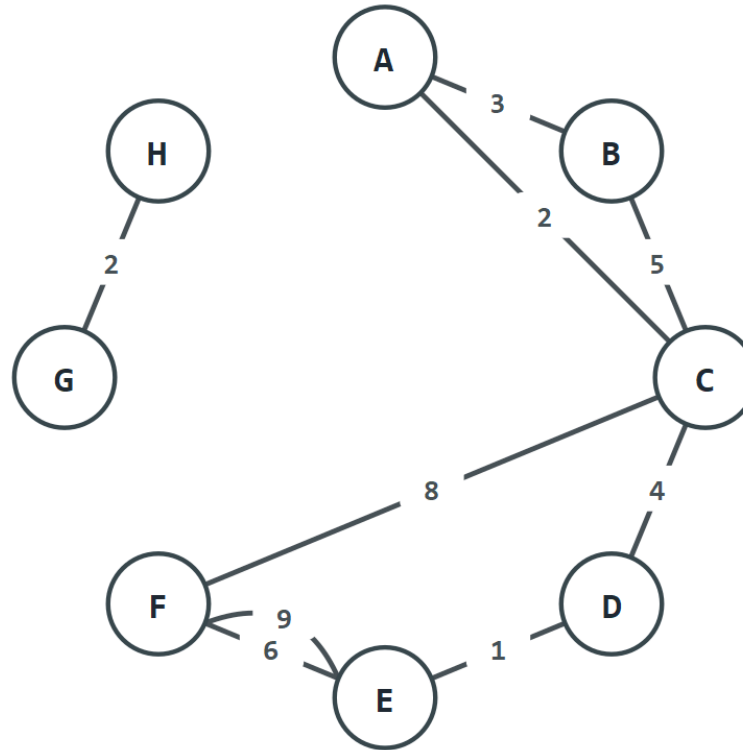
Seyrek ve yoğun

- **Seyrek** (sparse): E , V 'ye yakın -- düğüm başına az kenar
- **Yoğun** (dense): E , V^2 'ye yakın -- çiftlerin çoğu bağlı
- Bir yol haritası: seyrek (bir şehrin kavşak başına az yolu var)
- Küçük bir takımın "kim kimi takip ediyor" çizgesi: çoğu zaman yoğun

Bir çizgeye ne sorabiliriz?

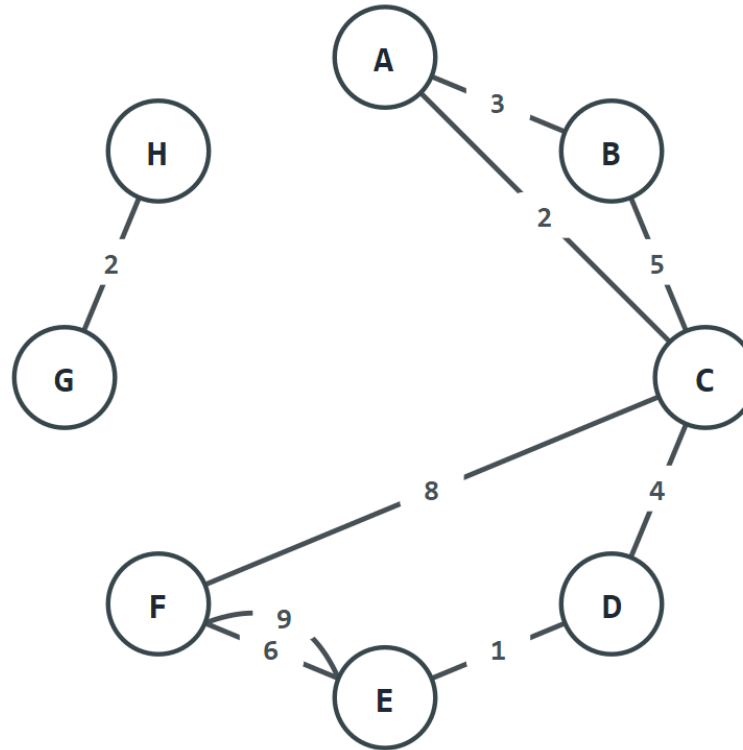
İşlem	Neyi yanıtlar
<code>has_edge(a, b)</code>	<code>a</code> ve <code>b</code> doğrudan bağlı mı?
<code>neighbors(v)</code>	<code>v</code> hangi düğümlere bağlı?
<code>degree(v)</code>	<code>v</code> 'ye kaç kenar geliyor?

Çizge terimleri, tek tek



Özet: 8 düğüm, 10 kenar, 2 bileşen, döngü var.

Uç durum — yönlü, iki döngü, bir öz-döngü



Özet: 8 düğüm, 10 kenar, 2 bileşen, döngü var.

Kod — Edge ve Graph yapıları

```
typedef struct Edge {  
    int to;           /* diğer uç */  
    int weight;       /* ağırlıksızsa 1 */  
    struct Edge *next;  
} Edge;  
  
typedef struct Graph {  
    Edge *adj[MAX_V]; /* düğüm başına bir liste */  
    int vertex_count;  
    int directed;     /* 0 ya da 1 */  
} Graph;
```

Kod — out_degree()

```
int out_degree(Graph *g, int v) {  
    int d = 0;  
    for (Edge *e = g->adj[v]; e; e = e->next)  
        d++;  
    return d;  
}
```

Kod — in_degree() (yalnız yönlü)

```
int in_degree(Graph *g, int v) {  
    int d = 0;  
    for (int u = 0; u < g->vertex_count; u++)  
        for (Edge *e = g->adj[u]; e; e = e->next)  
            if (e->to == v) d++;  
    return d;  
}
```

Karmaşıklık

- `out_degree` : bir listede yürü -- $O(v\text{'nin derecesi})$
- `in_degree` : **her** düğümün listesini tara -- $O(V + E)$
- Yönlü çizgeler "gelen"i "giden"den kesinlikle daha pahalı yapar

Sık yapılan hatalar

- Bir çoklu-çizgede **dereceyi komşu sayısıyla** karıştırmak
- Bir öz-döngünün dereceye **iki kez** eklendiğini unutmak
- Gelen ve giden derecenin hep eşit olduğunu varsaymak -- yalnız özel durumlarda

Mini soru

Yönlü bir çizgede **x** düğümünde bir öz-döngü var. Bu öz-döngü **x**'in gelen derecesini ne kadar artırır? Giden derecesini?

Yanıt

Her birini 1'er. $x > x$ öz-döngüsü bir kez x 'e giren kenar, bir kez x 'ten çıkan kenar olarak sayılır -- ikisi de 1 artar.

3. Çizgeyi Saklamak: Matris ve Liste

Başlangıç sorusu

`has_edge(a, b)` ve `neighbors(v)`, bu haftaki her algoritmanın soracağı iki soru. Her ikisini de yanıtlayan en yalın yapı nedir?

İki gösterim, genel bakış

	Komşuluk matrisi	Komşuluk listesi
Sakladığı	Bir $V \times V$ tablo	Düğüm başına bir liste
En iyi olduğu	Yoğun çizgeler	Seyrek çizgeler
<code>has_edge</code>	$O(1)$	$O(\text{derece})$

Komşuluk matrisi — fikir

$V \times V$ boyutunda bir tablo. `matrix[i][j]`,
`i` düğümünden `j` düğümüne bir kenar varsa
1 (ya da ağırlığı) tutar, yoksa **0**. Yönsüz
çizgiler hem `matrix[i][j]` hem `matrix[j][i]` 'yi doldurur.

Komşuluk matrisi, adım adım

	A	B	C	D	E	F	G
A	0	1	0	1	0	0	1
B	1	0	1	0	1	0	0
C	0	1	0	1	0	1	0
D	1	0	1	0	1	0	0
E	0	1	0	1	0	1	0
F	0	0	1	0	1	0	1
G	1	0	0	0	0	1	0

Bitti: matris tamamlandı. Bir kenarın var olup olmadığını sormak $matrix[i][j]$ 'ye bakmakla $O(1)$ sürer, ama V büyüdükçe tablo V^2 hücre kaplar (çoğu boş olsa bile).

Uç durum — tam çizge

	A	B	C	D	E	F	G
A	0	1	0	1	0	0	1
B	1	0	1	0	1	0	0
C	0	1	0	1	0	1	0
D	1	0	1	0	1	0	0
E	0	1	0	1	0	1	0
F	0	0	1	0	1	0	1
G	1	0	0	0	0	1	0

Bitti: matris tamamlandı. Bir kenarın var olup olmadığını sormak $matrix[i][j]$ 'ye bakmakla $O(1)$ sürer, ama V büyüdükçe tablo V^2 hücre kaplar (çoğu boş olsa bile).

Kod — add_edge() (matris)

```
void add_edge(int a, int b, int w, int dir) {  
    matrix[a][b] = w;  
    if (!dir)  
        matrix[b][a] = w; /* köşegene ayna */  
}
```

Kod — build_adjacency_matrix()

```
void build_adjacency_matrix(Edge *edges, int n,  
                           int directed) {  
    for (int i = 0; i < MAX_V; i++)  
        for (int j = 0; j < MAX_V; j++)  
            matrix[i][j] = 0;  
    for (int k = 0; k < n; k++)  
        add_edge(edges[k].a, edges[k].b,  
                 edges[k].weight, directed);  
}
```

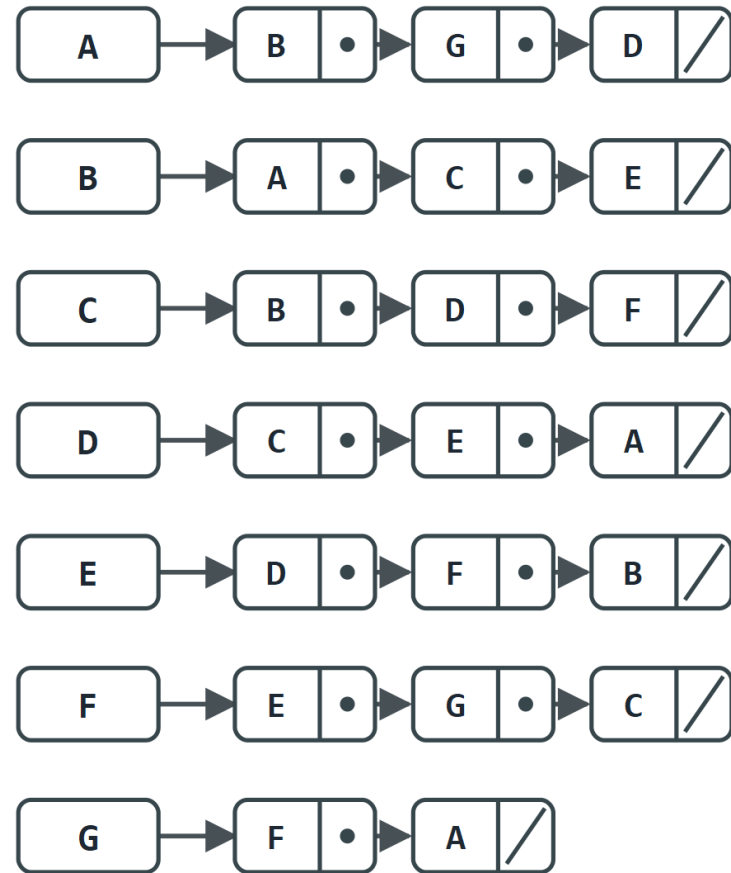
Karmaşıklık — matris

- `has_edge(a, b)` : tek bir bakış -- $O(1)$
- Bellek: yoğun ya da seyrek fark etmez, hep $O(V^2)$ hücre
- `neighbors(v)` : bütün bir satırı tara -- $O(V)$, v'nin 1 kenarı olsa bile

Komşuluk listesi — fikir

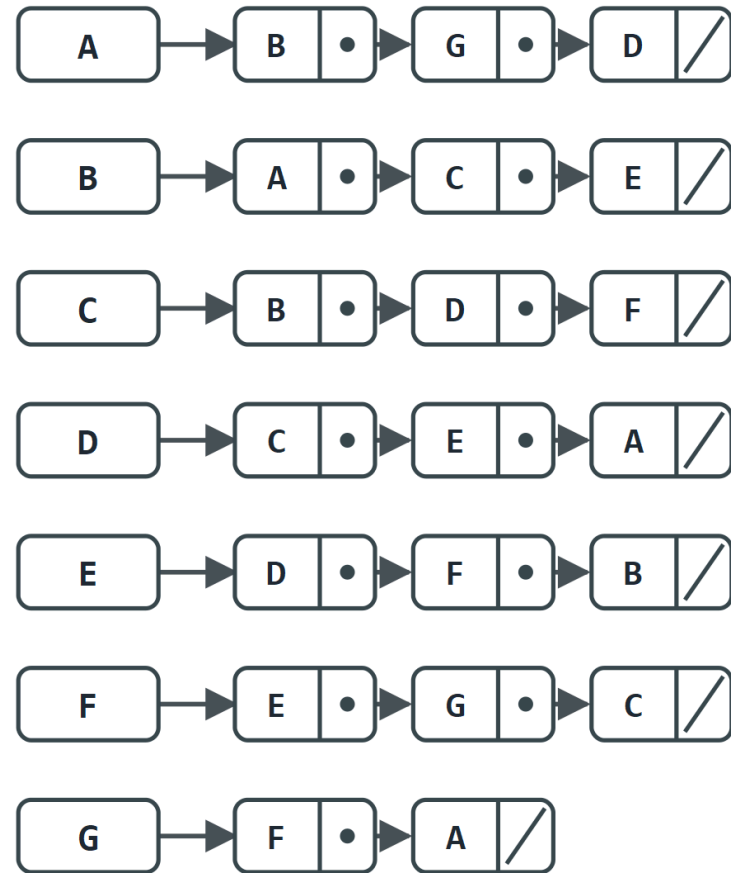
Her düğüm için bir bağlı liste, yalnızca gerçek komşularını tutar. Yönsüz **A-B** kenarı **B** 'yi **A** 'nın listesine **ve** **A** 'yı **B** 'nin listesine ekler -- iki düğüm, iki liste.

Komşuluk listesi, adım adım



Bitti: toplam 20 liste düğümü. Komşuluk listesi yalnız GERÇEK kenarlar kadar yer kaplar ($O(V + E)$); seyrek çizgelerde matristen çok daha tutumludur.

Uç durum — yönlü, boş bir liste



Bitti: toplam 20 liste düğümü. Komşuluk listesi yalnız GERÇEK kenarlar kadar yer kaplar ($O(V + E)$); seyrek çizgelerde matristen çok daha tutumludur.

Kod — append() (liste)

```
void append(int v, int neighbour) {  
    AdjNode *n = malloc(sizeof(AdjNode));  
    n->to = neighbour;  
    n->next = NULL;  
    if (adj[v] == NULL) { adj[v] = n; return; }  
    AdjNode *cur = adj[v];  
    while (cur->next) cur = cur->next;  
    cur->next = n;  
}
```

Kod — add_edge() (liste)

```
void add_edge(int a, int b, int directed) {  
    append(a, b);  
    if (!directed && a != b)  
        append(b, a);  
}
```

Karmaşıklık — liste

- Bellek: $O(V + E)$ -- yalnız gerçek kenarlar yer kaplar
- `neighbors(v)` : kendi listesinde yürü -- $O(v\text{'nin derecesi})$
- `has_edge(a, b)` : `a` 'nın listesinde `b` 'yi ara -- $O(\text{derece})$

Bellek ve zaman, yan yana

	Matris	Liste
Bellek	$O(V^2)$	$O(V + E)$
has_edge	$O(1)$	$O(\text{derece})$
neighbors	$O(V)$	$O(\text{derece})$

Seyrek çizgeler: listeyi seçin

- Bir yol haritası, web, bir arkadaşlık çizgesi: hepsi **seyrek**
- E , V^2 'nin çok altında, V 'ye yakın
- Listenin $O(V + E)$ belleği küçük kalır; matris $O(V^2)$ boşa gider
- Bölüm 4'ten sonraki her algoritma **listeyi** kullanıyor

Yoğun çizgeler: matris kazanabilir

- Küçük, yoğun bağlı bir çizge: E , V^2 'ye yakın
- Matrisin "boşa giden" $O(V^2)$ belleği neredeyse hiç boşa gitmez
- $O(1)$ `has_edge` , bellekten tasarruftan daha önemli olabilir
- Kural: küçük ve yoğunsa → matris; seyrek ya da büyükse → liste

Sık yapılan hatalar

- Büyük, seyrek bir çizge için matris kullanmak -- devasa bellek israfı
- Listenin bir yönsüz kenar için **iki** ekleme gerektirdiğini unutmak
- **Yönlü** bir çizgede `matrix[a][b]` 'yi simetrik okumak

Mini soru

Bir çizgede 1.000 düğüm, yalnızca 2.000 kenar var. Komşuluk matrisi kabaca kaç hücreye ihtiyaç duyar? Bu çizge seyrek mi?

Yanıt

Yaklaşık 1.000.000 hücre (1000^2), yalnızca 2.000 gerçek kenar için -- neredeyse hepsi boşa gidiyor. Evet, **çok seyrek**: burada açık seçim bir komşuluk listesi.

4. Genişlik Öncelikli Arama (BFS)

Başlangıç sorusu

Bir kişiden başlayarak, doğrudan arkadaşları kimler? Arkadaşlarının arkadaşları? Herkesi, **en yakından başlayarak** ziyaret et -- nasıl?

Kısa bir tarihçe

- **1959** — Edward F. Moore, "Labirentte en kısa yol" makalesini yayımlar
- Kısa süre sonra, bağımsız olarak, farklı alanlarda yeniden bulundu
- Buradaki kuyruk tabanlı yöntem, esasen Moore'un algoritmasıdır
- Bugün pratikte en çok kullanılan çizge algoritmalarından biri

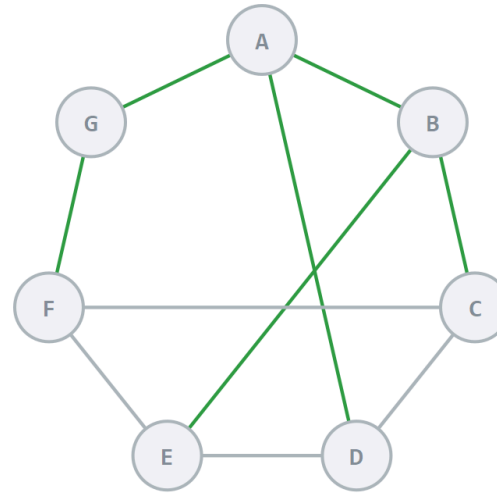
Sezgi — bir gölete atılan taş

- Bir taş at: ilk dalga en yakın noktalara ulaşır
- Sonraki dalga, bir adım daha uzaktaki her şeye ulaşır
- BFS bir çizgeyi aynı şekilde dolaşır -- halka halka, dışarı doğru
- Buradaki her "halka" bir **seviye** (level) olarak adlandırılır

BFS fikri

- Bir **kuyruk** (FIFO) kullan -- Hafta 3'ün yapısı, artık düğüm tutuyor
- Başlangıç düğümü: kuyruğa ekle, ziyaret edildi işaretle, seviye 0
- Bir düğümü çıkar, ziyaret edilmemiş komşularını ekle, seviye + 1
- Ziyaret edilen kenarlar bir **BFS ağacı** oluşturur, kökü başlangıç

BFS, adım adım



seviye[] =

A	B	C	D	E	F	G
0	1	2	1	2	2	1

kuyruk =

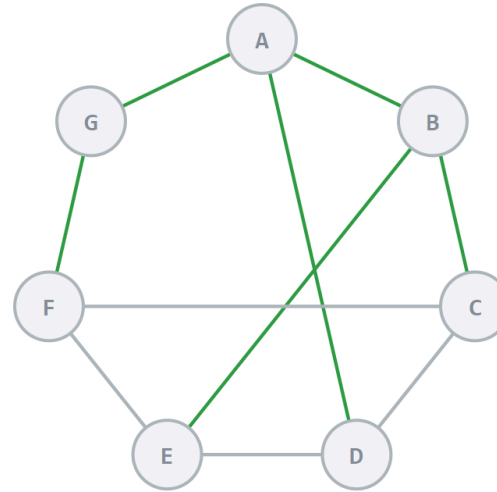
--	--	--	--	--	--	--

ziyaret sırası =

A	B	D	G	C	E	F
---	---	---	---	---	---	---

Kuyruk boş, bitti. Tam sıra: A, B, D, G, C, E, F -- her düğüme köke en az kenarla giden yoldan ulaşıldı.

Uç durum — bağlı olmayan bir çizge



seviye[] =

A	B	C	D	E	F	G
0	1	2	1	2	2	1

kuyruk =

--	--	--	--	--	--	--

ziyaret sırası =

A	B	D	G	C	E	F
---	---	---	---	---	---	---

Kuyruk boş, bitti. Tam sıra: A, B, D, G, C, E, F -- her düğüme köke en az kenarla giden yoldan ulaşıldı.

Kod — enqueue / dequeue

```
void enqueue(int v) {  
    queue_data[rear] = v;  
    rear = (rear + 1) % MAX_V;  
    count++;  
}  
int dequeue(void) {  
    int v = queue_data[front];  
    front = (front + 1) % MAX_V;  
    count--;  
    return v;  
}
```

Kod — bfs() ana döngüsü

```
visited[start] = 1;
level_of[start] = 0;
enqueue(start);
while (count > 0) {
    int u = dequeue();
    for (Edge *e = g->adj[u]; e; e = e->next) {
        if (!visited[e->to]) {
            visited[e->to] = 1;
            level_of[e->to] = level_of[u] + 1;
            parent_of[e->to] = u;
            enqueue(e->to);
        }
    }
}
```

Karmaşıklık

- Her düğüm bir kez eklenir, bir kez çıkarılır -- $O(V)$
- Her kenar en fazla iki kez incelenir (her uçtan bir kez) -- $O(E)$
- Toplam: $O(V + E)$ -- her liste işleminde gördüğümüz aynı sınır

BFS ağacı ve en kısa yollar

- BFS ağacının kök-düğüm yolu **en az kenarı** kullanır
- `level_of[v]`, başlangıçtan tam olarak bu en kısa kenar sayısıdır
- Bu yalnızca **ağırlıksız** çizgelerde geçerlidir -- Bölüm 8 bunu doğrudan kullanır
- Ağırlıklı en kısa yollar Dijkstra algoritmasını gerektirir, Hafta 9

Uygulamalar

- "Arkadaşımın arkadaşı" -- Bölüm 4'ün sosyal ağ motivasyonu
- Web'i seviye seviye taramak, önce en yakın sayfalar
- Bulmaca çözücüler: bir labirenti ya da bir kayan bulmacayı en az hamleyle çözmek
- Ağ yayını: her makineye en az sıçramada ulaşmak

Sık yapılan hatalar

- Kuyruk yerine bir **yığın** kullanmak -- sessizce derinlik öncelikliye döner
- Bir düğümü **ekleme anında** değil çıkarma anında ziyaret edildi işaretlemek
- Zaten ziyaret edilmiş bir düğümü yeniden eklemek -- iş kaybı, döngü riski

Mini soru

Ağırlıksız bir çizgede, A'dan başlayan BFS **B**'ye seviye 2 veriyor. A'dan B'ye en kısa yolda kaç kenar var?

Yanıt

Tam olarak 2 kenar. BFS'in seviyesi,
başlangıçtan en az kenar sayısı olarak tanımlanır
-- bu, tam olarak en kısa yol kenar sayısıdır.

5. Derinlik Öncelikli Arama (DFS) — Özyinelemeli

Başlangıç sorusu

Bir labirentte çok sayıda dallanan yol var.
Her yönü azar azar keşfetmek yerine, geri
dönmeden önce **tek** bir yola sonuna kadar
bağlı kalsanız ne olur?

Sezgi — bir labirent, önce derine in

- Bir yön seç, gidebildiği kadar yürü
- Çıkamaz sokak mı? Son seçime geri dön, sıradaki yönü dene
- "Geri dönmek", bir **yığının** (ya da özyinelemenin) sizin için yaptığı şey
- Genişlik değil derinlik öncelikli -- bir dal, tamamen, sonrakinden önce

Renkler ve zamanlar

- **Beyaz** (0): henüz keşfedilmedi
- **Gri** (1): keşfedildi, komşuları hâlâ araştırılıyor
- **Siyah** (2): bitti, her komşu araştırıldı
- `disc[v]` / `fin[v]` : her renk değişiminin gerçekleştiği saat tıkırtısı

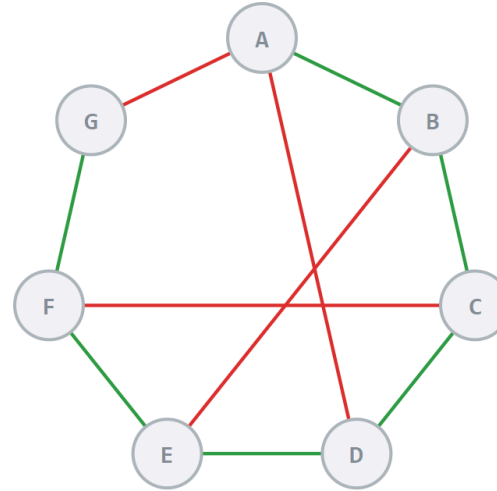
Kenar türleri

Tür	Anlamı
Ağaç kenarı	Yeni, beyaz bir düğüme götürür
Geri kenar (back edge)	Gri bir ataya götürür -- bir döngü!
İleri / çapraz	Bitmiş, siyah bir düğüme götürür (yalnız yönlü)

çağrı yığının =

Bitti. Ziyaret sırası: A, B, C, D, E, F, G.

Uç durum — dört kenar türü bir arada



keşif/bitiş[] =

A	B	C	D	E	F	G
1/14	2/13	3/12	4/11	5/10	6/9	7/8

çağrı yığını =

--	--	--	--	--	--	--

ziyaret sırası =

A	B	C	D	E	F	G
---	---	---	---	---	---	---

Bitti. Ziyaret sırası: A, B, C, D, E, F, G.

Kod — dfs_visit(): keşif

```
void dfs_visit(Graph *g, int u) {
    color_of[u] = 1;          /* gri */
    disc_time[u] = ++clock_;
    for (Edge *e = g->adj[u]; e; e = e->next) {
        int v = e->to;
        /* ... v'ye giden kenarı sınıfla ... */
    }
    color_of[u] = 2;          /* siyah */
    fin_time[u] = ++clock_;
}
```

Kod — bir kenarı sınıflamak

```
if (color_of[v] == 0) {  
    parent_of[v] = u;  
    dfs_visit(g, v);    /* ağaç kenarı */  
} else if (color_of[v] == 1) {  
    /* geri kenar: v bir ata */  
} else {  
    /* v siyah: ileri ya da çapraz */  
}
```

Kod — bileşen başına bir ağaç

```
void dfs(Graph *g) {  
    for (int i = 0; i < g->vertex_count; i++)  
        color_of[i] = 0;  
    for (int i = 0; i < g->vertex_count; i++)  
        if (color_of[i] == 0)  
            dfs_visit(g, i);  
}
```

Karmaşıklık

- Her düğüm beyaz $\rightarrow$ gri $\rightarrow$ siyah olarak **bir kez** renklenir -- $O(V)$
- Her kenar tam bir kez incelenir (yönsüzse iki kez) -- $O(E)$
- Toplam: **$O(V + E)$** -- BFS'in sınırıyla aynı

Özyineleme derinliği: gizli bir risk

- Her `dfs_visit` çağrısı **çağrı yığının**a bir çerçeve ekler
- Uzun bir zincir çizge, `v` çağrı kadar derinleşebilir
- Çok büyük ya da çok "ipliksi" bir çizge bu yığının **taşırabilir**
- Bölüm 6, tam olarak bunu önlemek için DFS'i açık bir yığınla yeniden kuruyor

Sık yapılan hatalar

- Doğrudan kendi **ebeveynine** geri giden kenarı atlamayı unutmak
- **Yönlü** bir çizgede her ağaç-dışı kenarı "döngü" saymak
- `color_of` 'u sıfırlamadan çalıştırmalar arasında yeniden kullanmak

Mini soru

Yönsüz bir çizgede, DFS şu anda **gri** olan bir düğüme giden bir kenar buluyor. Bu, çizge hakkında ne söyler?

Yanıt

Çizgede bir döngü var. Gri bir düğüm hâlâ çağrı yığınındadır -- şu anki düğümün bir **atasıdır**, yani bu kenar bir geri kenardır.

6. DFS — Yinelemeli (Açık Yığın)

Başlangıç sorusu

Bölüm 5 bir uyarıyla bitti: derin özyineleme çağrı yığınını taşıyabilir. *Tam olarak aynı* dolaşma özyinelemesiz yazılabilir mi?

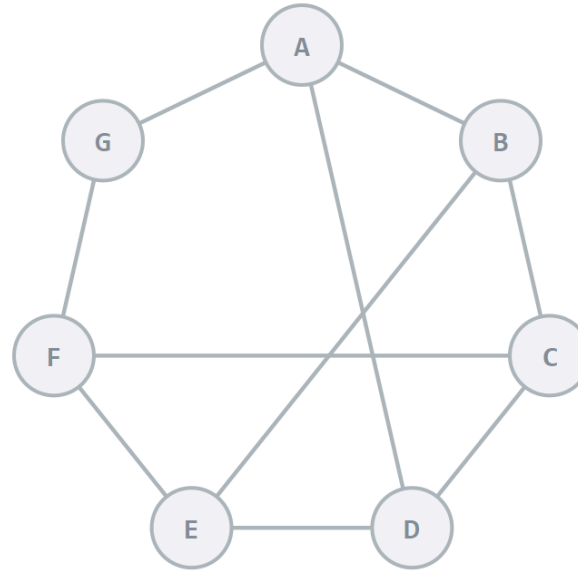
Neden yinelemeliye geçmeli

- Özyinelemenin çağrı yığınının sabit bir boyutu var, işletim sistemi belirler
- **Kendi** yığınımız (Hafta 3'ün dizi tabanlısı) ihtiyaca göre boyutlanabilir
- Aynı algoritma, aynı ziyaret sırası -- yalnızca kayıt tutma yer değiştiriyor
- Toplamda bellek tasarrufu yok, yalnızca daha güvenli bir yere taşınıyor

Ters sıra hilesi

- Komşular, özyinelemedeki gibi **alfabetik** sırayla ziyaret edilmeli
- Alfabetik sırayla eklemek onları **tersten** çıkarır
- Çözüm: komşuları **ters** alfabetik sırayla yığına ekle
- Tersten eklenen sıra, çıkarılınca özgün sırayı geri verir

Yinelemeli DFS, adım adım

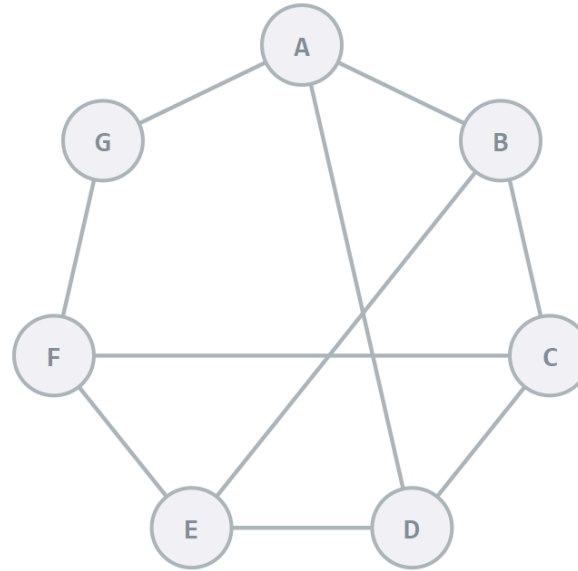


yığın =

ziyaret sırası =

Yığın boş, bitti. Ziyaret sırası: A, B, C, D, E, F, G -- özyinelemeli sürümle tamamen aynı.

Uç durum — bir DFS ormanı



yığın =

ziyaret sırası =

Yığın boş, bitti. Ziyaret sırası: A, B, C, D, E, F, G -- özyinelemeli sürümle tamamen aynı.

Kod — push / pop

```
void push(int v) { stack_data[++top] = v; }  
int pop(void)   { return stack_data[top--]; }
```

Kod — dfs_iterative() ana döngüsü

```
push(start);
while (top >= 0) {
    int u = pop();
    if (visited[u]) continue; /* eskimiş kayıt */
    visited[u] = 1;
    for (int i = deg[u] - 1; i >= 0; i--)
        if (!visited[adj[u][i]])
            push(adj[u][i]);
}
```

Karmaşıklık

- Her düğüm sınırlı sayıda eklenir ve çıkarılır -- $O(V)$
- Her kenar bir kez incelenir (yönsüzse iki kez) -- $O(E)$
- Toplam: $O(V + E)$ -- özyinelemeli sürümden ne daha iyi ne daha kötü

Özyinelemeli ve yinelemeli, yan yana

	Özyinelemeli (Böl. 5)	Yinelemeli (Böl. 6)
Yığın	Çağrı yığını	Kendi dizimiz
Ziyaret sırası	Alfabetik	Alfabetik (ters eklenmiş)
Derin çizge riski	Taşabilir	Dizi boyutuyla sınırlı

Sık yapılan hatalar

- Komşuları ters yerine **düz** alfabetik sırayla eklemek
- Eskimiş-kayıt kontrolünü unutmak -- çift ziyaret, yanlış disc/fin
- Çıkarılan, ziyaret edilmemiş bir düğümün hep yeni keşfedildiğini varsaymak

Mini soru

Bir düğüm, hiçbir çıkarma olmadan, iki farklı komşudan **iki kez** yığına ekleniyor. Gerçekte kaç kez ziyaret edilir?

Yanıt

Bir kez. İlk `pop()` onu ziyaret eder ve ziyaret edildi işaretler; ikinci `pop()`, `visited[u]` 'yu zaten doğru bulup atılır.

7. Bağlı Bileşenler

Başlangıç sorusu

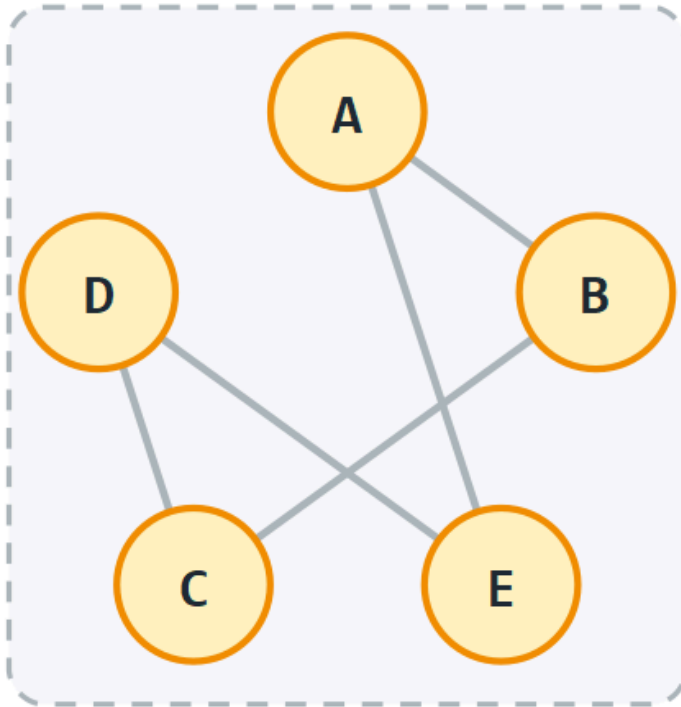
Bir çizge tek bir bağlı parça olmayabilir --
birkaç ayrı parçadan oluşabilir. Kaç tane?
Hangi düğüm hangi parçaya ait?

Fikir

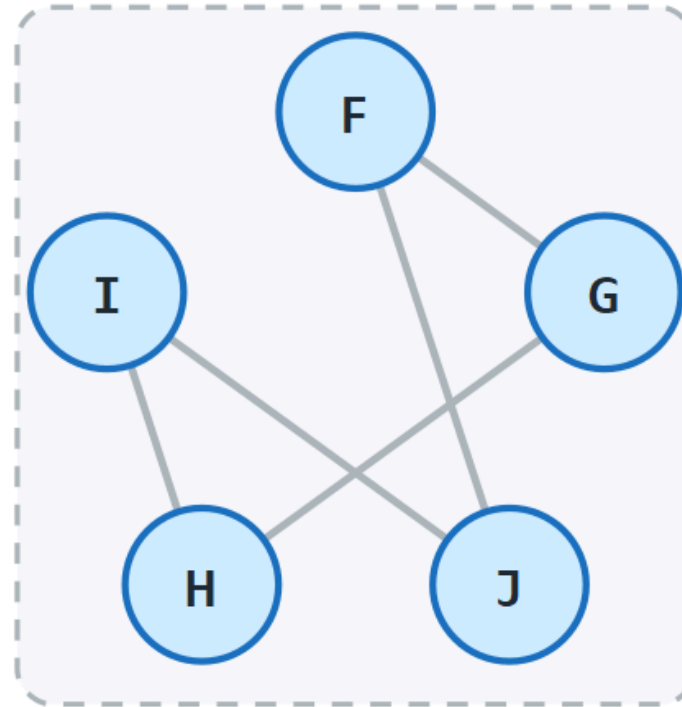
- Ziyaret edilmemiş her düğüm, yeni bir kimlikle **yeni** bir BFS başlatır
- O BFS, **erişebildiği her şeyi** aynı kimlikle etiketler
- Kuyruk boşalınca, ziyaret edilmemiş bir sonraki düğüm başlar
- Yön **göz ardı edilir** -- bu, **zayıf** bağılılığı bulur

Bağlı bileşenler, adım adım

bileşen 1



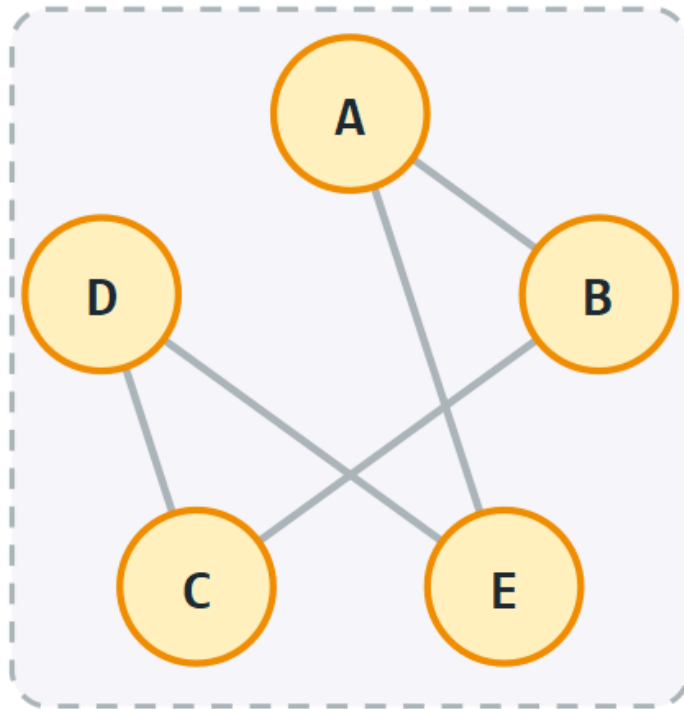
bileşen 2



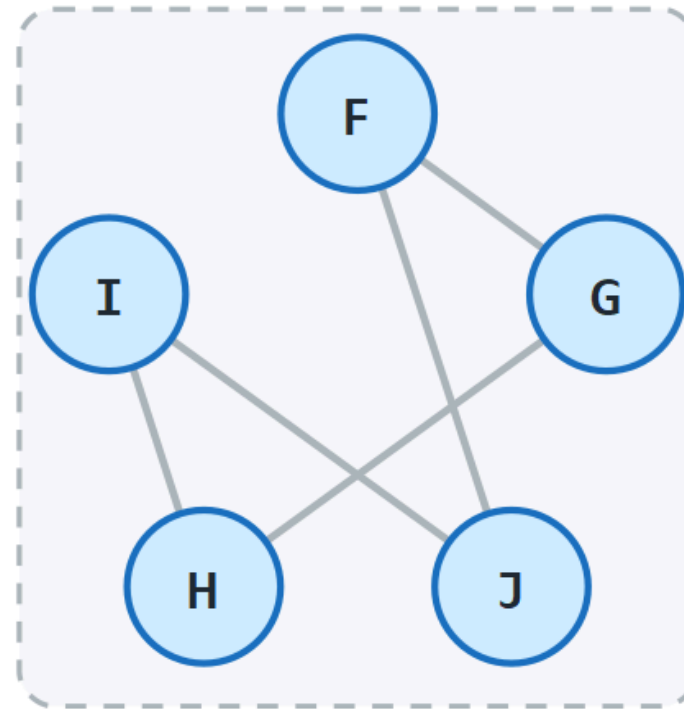
Bitti: 2 bağlı bileşen.

Uç durum — birçok küçük bileşen

bileşen 1



bileşen 2



Bitti: 2 bağlı bileşen.

Kod — bfs_label()

```
void bfs_label(Graph *g, int start, int id) {  
    int q[MAX_V], front = 0, rear = 0;  
    comp_of[start] = id;  
    q[rear++] = start;  
    while (front < rear) {  
        int u = q[front++];  
        for (Edge *e = g->adj[u]; e; e = e->next)  
            if (comp_of[e->to] == -1) {  
                comp_of[e->to] = id;  
                q[rear++] = e->to;  
            }  
    }  
}
```

Kod — count_components()

```
int count_components(Graph *g) {  
    int next_id = 0;  
    for (int i = 0; i < g->vertex_count; i++)  
        comp_of[i] = -1;  
    for (int i = 0; i < g->vertex_count; i++)  
        if (comp_of[i] == -1)  
            bfs_label(g, i, next_id++);  
    return next_id;  
}
```

Karmaşıklık

- Her düğüm tam olarak bir `bfs_label` çağrısına girer -- $O(V)$
- Her kenar, tüm çağrılar boyunca toplamda en fazla iki kez incelenir -- $O(E)$
- Toplam: $O(V + E)$ -- bileşen başına bir kez çalıştırılan tek bir BFS'in maliyeti

Uygulamalar

- Sosyal ağlar: hangi arkadaş çevreleri var, kim hangisinde
- Görüntü işleme: hangi piksellerin tek bir bağlı bölge oluşturduğu
- Ağ: hangi makinelerin şu an birbirine erişebildiği
- Bulmaca oyunları: "flood fill" -- boya kovası aracı, esasen

Sık yapılan hatalar

- Burada yönün **göz ardı edildiğini** unutmak -- bu zayıf bağlılıktır
- `comp_of` değerlerini sıfırlamadan ayrı çizgeler arasında yeniden kullanmak
- Tek düğümlü bir çizgenin **sıfır** bileşeni olduğunu varsaymak -- bir tane vardır

Mini soru

Yönlü bir çizgede $A \rightarrow B$ kenarı var ama $B \rightarrow A$ yok. Zayıf anlamda, A ve B aynı bileşende mi?

Yanıt

Evet. Zayıf bağıllık yönü tamamen göz ardı eder, o yüzden yalnızca $A > B$ bile A ve B 'yi aynı bileşene koymaya yeter.

8. Ağırlıksız Çizgelerde En Kısa Yol

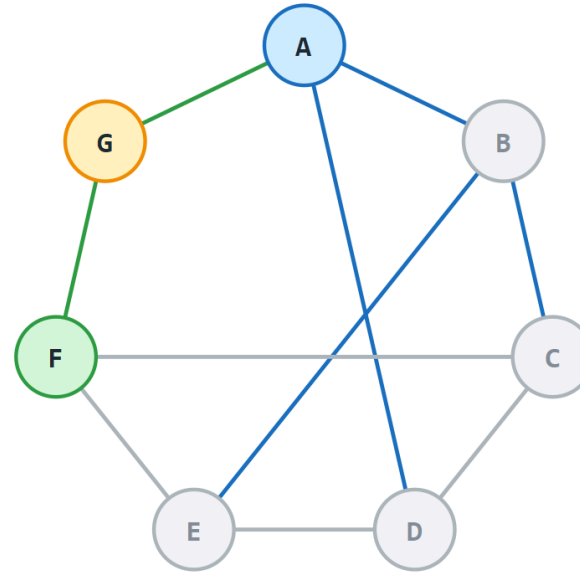
Başlangıç sorusu

BFS zaten **A** 'dan her düğüme en az kaç kenarla gidileceğini biliyor. **Tam güzergâhı** da -- yalnızca uzaklığı değil -- bildirebilir mi?

Fikir

- **s** 'den BFS çalıştır, her düğümün her zamanki gibi **ebeveynini** kaydet
- **t** 'ye giden yolu bulmak için: **t** 'den başla, **ebeveyn** i geriye izle
- **s** 'ye ulaşınca dur -- sonra toplanan listeyi **tersine çevir**
- Sonuç, kenar sayısına göre en kısa **s - t** yoludur

BFS ile yol bulma, adım adım



ebeveyn[] =

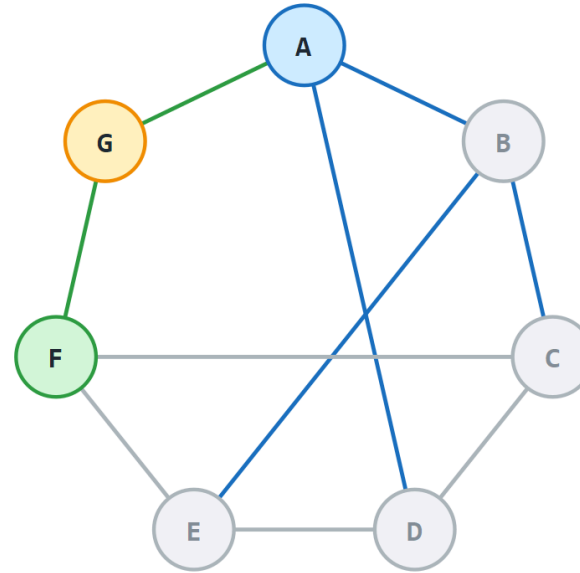
A	B	C	D	E	F	G
	A	B	A	B	G	A

kuyruk =

--	--	--	--	--	--	--

Ebeveyn işaretçileri F'den A'ye geriye izlenir: A → G → F. Uzunluk (kenar sayısı): 2.

Uç durum — hiçbir yol yok



ebeveyn[] =

A	B	C	D	E	F	G
	A	B	A	B	G	A

kuyruk =

--	--	--	--	--	--	--

Ebeveyn işaretçileri F'den A'ye geriye izlenir: $A \rightarrow G \rightarrow F$. Uzunluk (kenar sayısı): 2.

Kod — bfs_shortest_path(): arama

```
visited[s] = 1;
queue_data[rear++] = s;
while (front < rear) {
    int u = queue_data[front++];
    for (Edge *e = g->adj[u]; e; e = e->next)
        if (!visited[e->to]) {
            visited[e->to] = 1;
            parent_of[e->to] = u;
            queue_data[rear++] = e->to;
        }
}
```

Kod — yolu geriye izlemek

```
if (!visited[t]) return -1;      /* yol yok */
int len = 0, v = t;
while (v != s) {
    path_out[len++] = v;
    v = parent_of[v];
}
path_out[len++] = s;
/* ... path_out[0..len)'i yerinde tersine çevir ... */
return len - 1;                 /* kenar sayısı */
```

Karmaşıklık

- BFS'in kendisi: Bölüm 4'teki gibi tam olarak $O(V + E)$
- Yolu geriye izlemek: $O(\text{yol uzunluğu})$, en fazla $O(V)$
- Toplam: yine $O(V + E)$ -- geriye izleme hiçbir zaman baskın olmaz

Birden çok en kısa yol olabilir

- BFS **bir** en kısa yol bildirir, mutlaka tek yol değildir
- Farklı bir komşu ziyaret sırası, eşit uzunlukta farklı bir yol üretebilir
- Bütün doğru en kısa yollar aynı **uzunluğu** paylaşır, aynı güzergâhı değil

Önizleme: ağırlıklı çizgeler Dijkstra ister

- Bu bölüm yalnızca her kenarın gizlice ağırlığı 1 olduğu için çalışıyor
- **Ağırlıklı** bir çizge kenar sayısını değil gerçek mesafeleri karşılaştırmalı
- **Dijkstra algoritması** (Hafta 9) bunu bir **öncelik kuyruğuyla** çözer
- O öncelik kuyruğu, Hafta 4'ün bir yığından (heap) kurduğu ta kendisi

Sık yapılan hatalar

- Yolu geriye izlemeden önce `visited[t]` 'yi kontrol etmeyi unutmak
- `ebeveyn` i geriye değil, `t` 'den `s` 'ye doğru ileriye izlemek
- Son tersine çevirmeyi unutmak -- yol `t` 'den `s` 'ye doğru yazdırılır

Mini soru

`s` 'den başlayan BFS `t` 'yi hiç ziyaret etmiyor.
`bfs_shortest_path` ne döndürmeli, ve neden?

Yanıt

-1. `t` hiç ziyaret edilmediyse hiç yol yok demektir -- `visited[t]` kontrolü, herhangi bir geriye izleme başlamadan tam olarak bu durumu yakalar.

Özet — terimler ve gösterimler

Fikir	Anahtar gerçek
Çizge	V düğüm, E kenar, yönlü ya da değil
Derece	Yönsüz: tek değer; yönlü: gelen ve giden
Matris	$O(V^2)$ bellek, $O(1)$ <code>has_edge</code>
Liste	$O(V + E)$ bellek, $O(\text{derece})$ <code>has_edge</code>

Özet — dolaşmalar ve uygulamalar

Fikir	Anahtar gerçek
BFS	Kuyruk, seviye sırası, ağırlıksız en kısa yol
DFS (özyinelemeli / yinelemeli)	Yığın, önce derin, her iki yolda da aynı sıra
Bileşenler	Tekrarlanan BFS, erişilebilir küme başına bir etiket
En kısa yol	BFS + ebeveyn işaretçileri, geriye izlenir

Büyük resim

Bir ağacın "tek kök, döngü yok" kuralını kaldırın, bir çizge elde edersiniz. İki yapı onu saklar; iki dolaşma, BFS ve DFS, onu keşfeder; ikisi de gerçek sorulara yanıt verir -- bileşenler, mesafe.

Öz-değerlendirme turu

Dört kısa soru. Yanıt bir sonraki slaytta görünmeden önce düşünün. Tam alıştırmalar ve on soruluk bir sınav hafta notlarında.

1. Bir çizgede 6 düğüm var ve tam çizge (her çift bağlı). Kaç kenarı var?

$6 * 5 / 2 = 15$ kenar -- çizge yönsüz olduğundan her çift bir kez sayılır.

2. BFS neden yığın yerine kuyruğa ihtiyaç duyar?

Bir kuyruğun FIFO sırası en yakın düğümleri önce ziyaret eder; bir yığın hemen derine giderdi, yani DFS olurdu.

3. DFS sırasında bir "geri kenar" bulmak çizge hakkında ne söyler?

Çizgede bir döngü vardır -- bir geri kenar her zaman çağrı yığınınında hâlâ olan gri bir ataya işaret eder.

4. Bağlı bileşenler yönlü bir çizgeyi sanki yönsüzmüş gibi ele alır. Buna ne denir?

Zayıf bağıllık -- yön göz ardı edilir; geriye bir yolu da gerektiren güçlü bağıllık daha zor, farklı bir sorudur.

Gelecek hafta

Hafta 6 — Arama ve Karma Tablolar

BFS ve DFS bir şey bulmak için bütün çizgeyi dolaşır. Ya bir öğeye doğrudan, neredeyse $O(1)$ sürede ulaşabilseydiniz?

Kaynaklar (1/2)

- Ders izlencesi, Hafta 5: `docs/syllabus/syllabus.tr.md`
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4. baskı. MIT Press
- Sedgewick, Wayne. *Algorithms*, 4. baskı. Addison-Wesley
- Euler, L. (1736). *Solutio problematis ad geometriam situs pertinentis*

Kaynaklar (2/2)

- Moore, E. F. (1959). "Labirentte en kısa yol"
- Hopcroft, J., Tarjan, R. (1973) -- DFS kenar sınıflaması
- Knuth. *The Art of Computer Programming, Cilt 1*, 3. baskı
- [williamfiset/Algorithms](#) · Programiz DSA